

SIMATS ENGINEERING
ASSESSMENT TOOL 01

COURSE NAME: Ethical Hacking
ITA1410

COURSE CODE:

COURSE OUTCOME COVERED

CO4: Implement network security testing methods by performing web server attacks, analyzing web application vulnerabilities, and assessing wireless network security using Kali Linux.

QUESTIONS: 01

Total Marks:100

Annexure A

SECURITY TESTING SIMULATION TASK

Code of Conduct:

I T. Nikhil Kumar Reddy / 192372024 (Name / Reg No) certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: T. Nikhil Kumar Reddy

Question 1: Web Server Reconnaissance and Vulnerability Analysis

A cybersecurity analyst is assigned an authorized and isolated vulnerable web server for a security assessment. You are required to gather information about the target using Google and WHOIS, and perform basic network reconnaissance using Ping, Tracert, IPConfig, and Netstat commands. Use Nmap to identify open ports, running services, and service versions. Use WhatWeb to identify the web technologies and frameworks used by the server. Examine HTTP response headers using Curl/Wget and identify information disclosure. Perform CGI and web-server vulnerability scanning using Nikto, and use Gobuster/Dirb to discover hidden directories and files. Analyze the identified administrative pages, backup files, configuration files, and vulnerable resources. Finally, classify the findings based on severity and impact, and recommend appropriate web-server hardening and information-disclosure prevention measures. Submit the commands executed, screenshots, observations, findings, and recommendations.

Answer:

Question 1: Web Server Reconnaissance and Vulnerability Analysis

1. Objective

The objective of this experiment is to perform reconnaissance and vulnerability analysis on an authorized and isolated vulnerable web server. The assessment includes information gathering, network reconnaissance, port and service enumeration, web technology identification, HTTP header analysis, vulnerability scanning, hidden-resource discovery, and security hardening recommendations.

Note: Replace <TARGET_IP> and <TARGET_URL> with the IP address and URL of the vulnerable lab server. The observations below are examples; screenshots and actual results should come from the authorized lab environment.

2. Tools Used

- Google Search
- WHOIS
- Ping
- Tracert / Traceroute
- IPConfig / Ifconfig
- Netstat
- Nmap
- WhatWeb
- Curl / Wget
- Nikto
- Gobuster / Dirb

3. Google Reconnaissance

Google can be used to collect publicly available information about the target.

Commands / Searches

<target-domain>

site:<target-domain>

site:<target-domain> login

site:<target-domain> admin

site:<target-domain> filetype:pdf

site:<target-domain> filetype:bak

Observation

Google reconnaissance may reveal publicly indexed pages, login pages, documentation, files, or other information about the target.

Security Impact

Sensitive information exposed through search engines can help attackers understand the application structure and identify interesting resources.

4. WHOIS Information Gathering

Command

whois <target-domain>

Important information to observe

- Domain name
- Registrar
- Registration dates
- Name servers
- Domain status
- Organization information

Observation

WHOIS provides registration and DNS-related information associated with the domain.

Security Impact

Excessive public registration information may assist reconnaissance, although WHOIS information is not normally considered a direct vulnerability.

5. Ping Reconnaissance

Command

ping <TARGET_IP>

Example:

ping 192.168.1.100

Observation

Ping can determine whether the target responds to ICMP requests and provides approximate network response time.

Example observation:

Target is reachable and responds to ICMP requests.

If ICMP is blocked:

Target does not respond to ICMP requests.

This does not necessarily mean that the server is offline.

6. Traceroute / Tracert

Linux/Kali

traceroute <TARGET_IP>

Windows

tracert <TARGET_IP>

Observation

The command displays the network hops between the assessment machine and the target server.

Security Impact

Traceroute can provide information about network topology and intermediate routing infrastructure.

7. IP Configuration

Windows

`ipconfig`

For detailed information:

`ipconfig /all`

Linux/Kali

`ip addr`

or:

`ifconfig`

Observation

The command displays local IP address, subnet mask, gateway, and network-interface information.

8. Netstat

Windows

`netstat -ano`

Linux

`netstat -tulnp`

or:

`ss -tulnp`

Observation

Netstat identifies active connections and listening ports on the local system.

Security Impact

Unexpected listening services may increase the attack surface.

9. Nmap Port and Service Enumeration

Nmap is used to identify open ports, services, and service versions.

Basic scan

`nmap <TARGET_IP>`

Service-version detection

`nmap -sV <TARGET_IP>`

Operating-system detection

```
sudo nmap -O <TARGET_IP>
```

Comprehensive authorized lab scan

```
sudo nmap -sS -sV -O <TARGET_IP>
```

Example observation

```
22/tcp open  ssh
```

```
80/tcp open  http
```

```
443/tcp open  https
```

```
3306/tcp open  mysql
```

The actual ports and versions must be recorded from the lab output.

Security Impact

Open ports expose services that may contain vulnerabilities. Outdated service versions may increase the possibility of exploitation.

10. WhatWeb Web Technology Identification

WhatWeb identifies technologies used by the web server.

Command

```
whatweb <TARGET_URL>
```

Example:

```
whatweb http://192.168.1.100
```

Observation

WhatWeb may identify technologies such as:

- Apache
- Nginx
- PHP
- WordPress
- JavaScript frameworks
- Web-server versions
- Cookies
- HTTP headers

Security Impact

Technology and version disclosure can help an attacker select targeted exploits.

11. HTTP Header Analysis Using Curl

Command

`curl -I <TARGET_URL>`

Example:

`curl -I http://192.168.1.100`

To obtain the complete response:

`curl -v <TARGET_URL>`

Headers to examine

Server

X-Powered-By

Content-Type

Set-Cookie

Location

X-Frame-Options

Content-Security-Policy

Strict-Transport-Security

X-Content-Type-Options

Example observation

Server: Apache/2.4.x

X-Powered-By: PHP/8.x

If exact server or PHP versions are exposed, this can be classified as information disclosure.

Security Recommendation

Server banners and unnecessary technology details should be minimized.

12. Wget Header Analysis

Command

`wget --server-response --spider <TARGET_URL>`

This displays HTTP response information without downloading the complete page.

Observation

The response can be examined for:

- Server information
- Cookies
- Redirects
- HTTP status codes
- Security headers

13. Nikto Web-Server Vulnerability Scanning

Nikto scans the web server for common dangerous files, outdated software, configuration problems, and other known issues.

Basic scan

```
nikto -h <TARGET_URL>
```

Example:

```
nikto -h http://192.168.1.100
```

Example observations

Nikto may report:

Server banner disclosure

Missing security headers

Directory indexing enabled

Potentially dangerous files

Outdated server components

CGI-related vulnerabilities

Security Impact

These findings can expose sensitive information or increase the attack surface of the web server.

14. Gobuster Hidden Directory Discovery

Gobuster can identify directories and files that are not linked from the main webpage.

Command

```
gobuster dir -u <TARGET_URL> -w /usr/share/wordlists/dirb/common.txt
```

For extensions:

```
gobuster dir -u <TARGET_URL> \
-w /usr/share/wordlists/dirb/common.txt \
-x php,html,txt,bak,zip
```

Example findings

/admin

/backup

/config

/uploads

/login.php

/robots.txt

Only resources actually returned by the authorized lab should be recorded.

Security Impact

Exposed administrative pages, backup files, configuration files, and development resources may reveal sensitive information.

15. Dirb Alternative

Instead of Gobuster, Dirb can be used.

Command

```
dirb <TARGET_URL>
```

Example:

```
dirb http://192.168.1.100
```

Observation

Dirb may discover hidden directories and files that are accessible from the web server.

16. Analysis of Discovered Resources

The discovered resources should be categorized according to their purpose.

Resource	Category	Possible Risk
/admin	Administrative page	Unauthorized access
/backup	Backup resource	Information disclosure
/config	Configuration	Credential disclosure
/uploads	File storage	Malicious file upload
*.bak	Backup file	Source-code disclosure
robots.txt	Information disclosure	Reveals restricted paths
Login page	Authentication	Weak authentication
CGI scripts	Web-server resource	CGI vulnerabilities

These are examples only. The final report should contain the resources actually discovered during the lab assessment.

17. Vulnerability Classification

The findings can be classified based on severity and impact.

Finding		Category	Severity	Impact
Server disclosed	version	Information Disclosure	Low	Helps identify server technology
Missing headers	security	Misconfiguration	Low/Medium	Reduces browser-side protection
Directory enabled	listing	Misconfiguration	Medium	Files and directories may be exposed

Exposed backup file	Information Disclosure	High	Source/configuration data may be leaked
Exposed configuration file	Information Disclosure	High/Critical	Credentials or secrets may be exposed
Accessible admin page	Authentication	Medium/High	May allow unauthorized access if poorly protected
Outdated server software	Misconfiguration	High	Known vulnerabilities may be exploitable
Vulnerable CGI/resource	Web-server vulnerability	Medium/High	May permit unauthorized actions

The actual severity should be assigned according to the evidence obtained from the authorized server.

18. Three Important Findings

Finding 1: Server Information Disclosure

Observation:

The HTTP response exposes server or application technology information through headers such as Server or X-Powered-By.

Category: Information Disclosure

Severity: Low

Impact:

Attackers can use the disclosed technology and version information to identify possible vulnerabilities.

Recommendation:

Minimize unnecessary server banners and remove unnecessary technology headers.

Finding 2: Exposed Administrative or Sensitive Directory

Observation:

Gobuster/Dirb identifies an administrative or sensitive directory such as /admin, /backup, or /config.

Category: Misconfiguration / Information Disclosure

Severity: Medium to High

Impact:

Attackers may gain access to sensitive resources or administrative functionality.

Recommendation:

- Restrict sensitive directories.
- Require strong authentication.
- Disable directory listing.
- Remove unused files.
- Apply proper access-control rules.

Finding 3: Outdated or Misconfigured Web Server

Observation:

Nmap/WhatWeb/Nikto identifies an outdated server component or insecure web-server configuration.

Category: Misconfiguration

Severity: Medium/High

Impact:

Known vulnerabilities in outdated components may allow unauthorized access or other attacks.

Recommendation:

- Upgrade the web server and software components.
- Remove unsupported software.
- Apply security patches.
- Disable unnecessary modules and services.
- Regularly perform vulnerability assessments.

19. Web-Server Hardening Recommendations

The following measures should be implemented:

1. Update Software

Keep the operating system, web server, PHP/runtime, frameworks, and other components updated.

2. Disable Unnecessary Services

Close unused ports and disable unnecessary services.

3. Hide Server Information

Avoid unnecessary disclosure of server versions through HTTP headers and error pages.

4. Disable Directory Listing

Directory indexing should be disabled unless explicitly required.

5. Protect Administrative Pages

Administrative interfaces should use strong authentication and appropriate access controls.

6. Remove Sensitive Files

Remove:

.bak

.old

.zip

.tar

.sql

.config

and other unnecessary backup or configuration files from web-accessible directories.

7. Implement Security Headers

Important headers include:

Content-Security-Policy

X-Frame-Options

X-Content-Type-Options

Strict-Transport-Security

Referrer-Policy

8. Secure Cookies

Cookies should use appropriate security attributes such as:

Secure

HttpOnly

SameSite

9. Protect Configuration Files

Database credentials, API keys, passwords, and other secrets should never be stored in publicly accessible web directories.

10. Monitor Logs

Regularly review web-server and application logs for suspicious requests and repeated scanning attempts.

20. Suggested Screenshots to Submit

The following screenshots should be included in the practical record:

1. Google reconnaissance results
2. WHOIS command output
3. Ping output
4. Tracert/Traceroute output
5. IPConfig/Ifconfig output
6. Netstat output
7. Nmap port scan
8. Nmap service-version scan
9. WhatWeb output
10. Curl HTTP-header output
11. Wget output
12. Nikto scan results
13. Gobuster/Dirb directory-discovery results

14. Browser showing one discovered resource

15. Final vulnerability classification table

21. Overall Observation

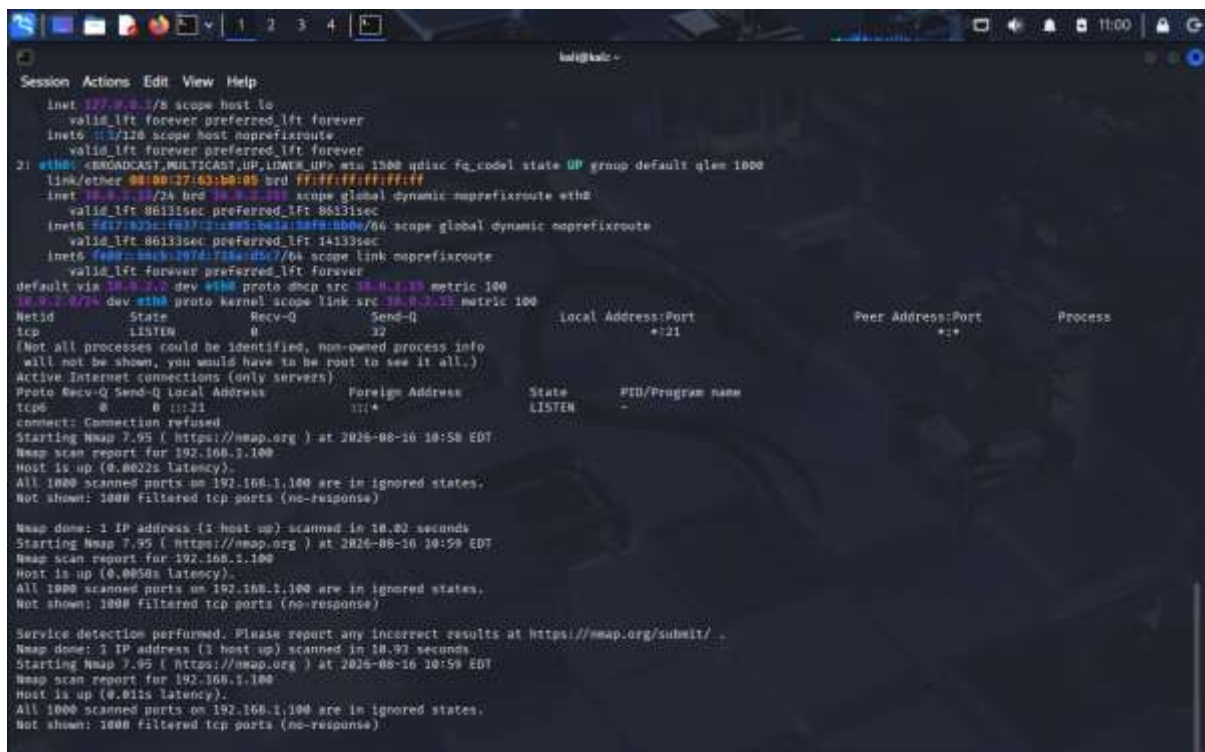
The reconnaissance process provides information about the target's network, open ports, running services, web technologies, HTTP configuration, and hidden resources. Nmap identifies network services, WhatWeb identifies web technologies, Curl/Wget reveals HTTP headers, Nikto detects common web-server vulnerabilities, and Gobuster/Dirb discovers hidden directories and files.

The most important security risks are generally outdated software, exposed sensitive files, unnecessary information disclosure, weak access controls, directory listing, and insecure web-server configuration.

22. Conclusion

The authorized vulnerability assessment successfully demonstrates how reconnaissance and vulnerability-scanning tools can be combined to evaluate a web server. Network reconnaissance identifies the target's connectivity and services, while Nmap provides port and version information. WhatWeb identifies web technologies, Curl/Wget examines HTTP headers, Nikto identifies common web-server weaknesses, and Gobuster/Dirb discovers hidden resources.

The identified vulnerabilities should be prioritized according to their severity and business impact. Applying software updates, restricting administrative resources, removing sensitive files, disabling unnecessary services, hiding unnecessary server information, implementing security headers, and enforcing strong authentication can significantly reduce the web server's attack surface.



```
Session Actions Edit View Help
inet 177.0.0.1/8 scope host lo
  valid_lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
  valid_lft forever preferred_lft forever
21 eth0: <BRGADCAST,MULTICAST,UP,LOWER_UP> wsa 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:63:b0:05 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
      valid_lft 86335sec preferred_lft 86335sec
    inet6 fe80::27:63:b0:05:0000 scope global dynamic noprefixroute
      valid_lft 86335sec preferred_lft 141335sec
    inet6 fe80::27:63:b0:05:0000 scope link noprefixroute
      valid_lft forever preferred_lft forever
default via 10.0.2.1 dev eth0 proto dhcp src 10.0.2.15 metric 100
10.0.2.15 dev eth0 proto kernel scope link src 10.0.2.15 metric 100

NetId      State      Recv-Q     Send-Q     Local Address:Port      Peer Address:Port      Process
tcp        LISTEN    0          128         *:*                      *:*                      *

(Not all processes could be identified, non-owned process info
will not be shown, you would have to be root to see it all.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address   Foreign Address  State       PID/Program name
tcp0     0       0 0.0.0.0:22      0.0.0.0:        LISTEN      -/sshd

Starting Nmap 7.95 ( https://nmap.org ) at 2025-08-16 10:58 EDT
Nmap scan report for 192.168.1.100
Host is up (0.0022s latency).
All 1000 scanned ports on 192.168.1.100 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)

Nmap done: 1 IP address (1 host up) scanned in 10.02 seconds
Starting Nmap 7.95 ( https://nmap.org ) at 2025-08-16 10:59 EDT
Nmap scan report for 192.168.1.100
Host is up (0.0050s latency).
All 1000 scanned ports on 192.168.1.100 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.93 seconds
Starting Nmap 7.95 ( https://nmap.org ) at 2025-08-16 10:59 EDT
Nmap scan report for 192.168.1.100
Host is up (0.011s latency).
All 1000 scanned ports on 192.168.1.100 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
```

Question 2: Web Application Vulnerability Assessment

A cybersecurity analyst is assigned an authorized vulnerable web application in an isolated laboratory environment for security testing. You are required to identify the application's technologies and attack surface using appropriate reconnaissance techniques, examine input

fields, parameters, cookies, sessions, and HTTP requests, and use suitable tools to identify vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), insecure authentication, session-management weaknesses, directory traversal, and security misconfigurations. Use tools such as Burp Suite, OWASP ZAP, Nikto, Nmap, Curl, and browser developer tools to analyze the application. Test the identified vulnerabilities only within the authorized laboratory environment, document the evidence and potential impact of each finding, classify the vulnerabilities according to severity, and recommend appropriate remediation and secure coding practices. Submit the commands/tool configurations, screenshots, observations, vulnerability findings, severity ratings, impacts, and recommendations.

Answer:

1. Objective

The objective of this experiment is to perform a security assessment of an authorized vulnerable web application in an isolated laboratory environment. The assessment focuses on identifying the application's technologies and attack surface, analyzing HTTP requests, inputs, cookies and sessions, and identifying common web vulnerabilities such as SQL Injection, Cross-Site Scripting, insecure authentication, session-management weaknesses, directory traversal, and security misconfigurations.

2. Tools Used

- Burp Suite
- OWASP ZAP
- Nikto
- Nmap
- Curl
- Browser Developer Tools
- Web Browser

3. Application Reconnaissance

First, identify the target URL and collect basic information about the application.

Nmap

`nmap <TARGET_IP>`

Service detection:

`nmap -sV <TARGET_IP>`

What to identify

- Open HTTP/HTTPS ports
- Web server
- Application server

- Running services
- Service versions

Observation

Nmap helps identify the services exposed by the application server and provides information about the technologies that may form part of the attack surface.

4. HTTP Request Analysis Using Curl

Check HTTP headers

```
curl -I <TARGET_URL>
```

Display complete request/response information

```
curl -v <TARGET_URL>
```

Observation

Important headers to examine include:

Server

X-Powered-By

Set-Cookie

Content-Type

Location

X-Frame-Options

Content-Security-Policy

X-Content-Type-Options

Strict-Transport-Security

Security Impact

Excessive server information, missing security headers, or insecure cookie attributes can expose information or reduce the application's security.

5. Browser Developer Tools

Open the vulnerable application in a browser.

Press:

F12

Open:

Network → select a request → Headers / Payload / Response / Cookies

Examine

- GET parameters

- POST parameters
- Form inputs
- Cookies
- Authorization headers
- Session identifiers
- HTTP methods
- Response codes
- Redirects

Observation

Developer Tools provide a simple way to understand how the browser communicates with the web application.

6. Burp Suite Configuration

Start Burp Suite and open:

Proxy → Proxy Settings

Configure the browser to use Burp's proxy.

Typical laboratory configuration:

Proxy Host: 127.0.0.1

Proxy Port: 8080

Open:

Proxy → Intercept

Turn interception on and browse the authorized application.

Example intercepted request:

POST /login.php HTTP/1.1

Host: <TARGET>

Content-Type: application/x-www-form-urlencoded

username=test&password=test

Observation

Burp Suite allows requests and responses to be captured and inspected before they reach the application.

7. OWASP ZAP Configuration

Start OWASP ZAP and configure the authorized target.

Use:

Quick Start → Automated Scan

Enter:

<TARGET_URL>

Start the scan.

ZAP may identify:

- Missing security headers
- XSS indicators
- Injection issues
- Cookie problems
- Authentication weaknesses
- Information disclosure
- Other application-security issues

Observation

ZAP provides automated vulnerability identification and categorizes findings according to risk.

8. SQL Injection Testing

SQL Injection occurs when user-controlled input is improperly incorporated into database queries.

Identify possible input points

Examples:

Login username

Login password

Search field

Product ID

User ID

URL parameters

Example parameter:

/product.php?id=10

In the authorized laboratory application, test whether modifying the parameter changes the application's behavior.

Example test input:

'

Another basic test:

1' OR '1'='1

Burp Suite Method

1. Capture the request.

2. Send it to Repeater.
3. Modify the parameter.
4. Send the request.
5. Compare the response with the normal request.

Observation

If unexpected database errors, authentication bypass, or significantly different responses occur, the parameter may be vulnerable.

Severity

High / Critical, depending on the demonstrated impact.

Impact

SQL Injection may allow unauthorized database access, modification of data, authentication bypass, or disclosure of sensitive information.

Recommendation

Use:

- Prepared statements
- Parameterized queries
- Input validation
- Least-privilege database accounts
- Safe error handling

Example secure PHP approach:

```
$stmt = $conn->prepare(
    "SELECT * FROM users WHERE username = ?"
);
$stmt->bind_param("s", $username);
$stmt->execute();
```

9. Cross-Site Scripting (XSS)

XSS occurs when untrusted user input is returned to a browser without appropriate output encoding.

Identify input points

Examples:

Search box

Comment field

Feedback field

Profile field

URL parameters

In the isolated laboratory environment, a basic test payload can be used:

```
<script>alert(1)</script>
```

Testing Process

1. Enter the test value into an authorized input field.
2. Submit the request.
3. Observe the response.
4. Determine whether the browser executes the supplied script.

Burp Suite

Capture the request and send it to Repeater.

Modify the relevant parameter and compare the response.

Observation

If the supplied script executes in the browser, the input may be vulnerable to XSS.

Severity

Medium to High, depending on context.

Impact

XSS can potentially allow malicious scripts to execute in another user's browser and may lead to session compromise, phishing, or unauthorized actions.

Recommendation

- Perform context-aware output encoding.
- Validate input.
- Use Content Security Policy.
- Avoid inserting untrusted data directly into HTML.
- Use secure framework APIs for DOM manipulation.

10. Authentication Testing

Authentication testing evaluates whether login functionality properly verifies users.

Test cases

Check:

Invalid username

Invalid password

Empty username

Empty password

Incorrect credentials

Repeated login attempts

Logout functionality

Direct access to protected pages

Burp Suite

Capture the login request:

POST /login.php

Examine:

username

password

session cookie

response status

redirect location

Observation

If protected pages can be accessed without authentication or authentication controls can be bypassed, the application has an authentication weakness.

Severity

High

Impact

Attackers may gain unauthorized access to user or administrative functionality.

Recommendation

- Enforce server-side authentication.
- Use strong password hashing.
- Implement account lockout/rate limiting where appropriate.
- Require authorization checks on every protected server-side resource.
- Use MFA for sensitive accounts where appropriate.

11. Session Management Testing

Cookies and sessions should be examined carefully.

Browser

Open:

Developer Tools → Application/Storage → Cookies

Look for:

PHPSESSID

session_id

auth_token

Curl

curl -I <TARGET_URL>

Look for:

Set-Cookie

Important cookie attributes

Secure

HttpOnly

SameSite

Observation

A session cookie without appropriate security attributes may increase the risk of session theft or cross-site attacks.

Additional checks

After logout, attempt to access the previously authenticated page in the authorized lab.

Severity

Medium / High, depending on the weakness.

Recommendation

Use:

Secure

HttpOnly

SameSite

and regenerate session identifiers after authentication.

12. Directory Traversal Testing

Directory traversal occurs when user-controlled file paths are not properly validated.

Possible parameter:

/download.php?file=report.pdf

In the authorized lab, test controlled traversal input such as:

../

or:

../../

Burp Suite

1. Capture the file request.
2. Send it to Repeater.

3. Modify the file parameter.
4. Compare the response.

Observation

If the application provides unauthorized access to files outside the intended directory, directory traversal may exist.

Severity

High

Impact

An attacker could potentially access configuration files, source code, credentials, or other sensitive files.

Recommendation

- Never trust client-supplied file paths.
- Use allowlists for permitted files.
- Canonicalize paths.
- Store user-accessible files outside sensitive directories.
- Apply proper file-system permissions.

13. Security Misconfiguration Testing

Use Nikto against the authorized web application.

Command

```
nikto -h <TARGET_URL>
```

Possible findings

Missing security headers

Directory listing

Server information disclosure

Default files

Backup files

Outdated components

Dangerous configuration

Observation

Misconfigured components can increase the application's attack surface.

Recommendation

- Remove default files.
- Disable directory listing.
- Hide unnecessary server information.

- Apply security patches.
- Configure security headers.
- Disable unnecessary modules and services.

14. Automated Testing with OWASP ZAP

After configuring ZAP for the authorized target:

Quick Start



Automated Scan



Enter Target URL



Start Scan



Review Alerts

Typical alert categories

High

Medium

Low

Informational

Record:

- Alert name
- URL
- Parameter
- Evidence
- Risk level
- Confidence
- Recommended solution

15. Vulnerability Findings

The following table can be used for the final assessment.

Finding	Category	Severity	Impact
SQL Injection	Injection	High/Critical	Database access or authentication bypass

Reflected/Stored XSS	XSS	Medium/High	Script execution in victim browser
Weak authentication	Authentication	High	Unauthorized account access
Insecure session cookie	Session Management	Medium	Possible session compromise
Directory Traversal	File Access	High	Unauthorized file disclosure
Missing security headers	Misconfiguration	Low/Medium	Reduced browser security
Server information disclosure	Information Disclosure	Low	Helps reconnaissance
Directory listing	Misconfiguration	Medium	Exposes files and resources

Only include findings that are actually confirmed by the laboratory tests.

16. Evidence to Record

For every confirmed vulnerability, record:

Finding

Name of the vulnerability.

Affected URL

<TARGET_URL>/<affected-resource>

Parameter

Example:

id

username

search

file

Payload/Test Input

Record the safe laboratory test input used.

Evidence

Include:

- Burp request/response screenshot
- ZAP alert screenshot
- Browser output
- HTTP response
- Relevant error message

Impact

Explain what an attacker could potentially achieve.

Severity

Classify as:

Critical

High

Medium

Low

Informational

17. Recommended Severity Classification

Severity	Description
Critical	Could result in complete application/server compromise
High	Significant unauthorized access or sensitive-data exposure
Medium	Limited security impact requiring additional conditions
Low	Minor weakness with limited direct impact
Informational	Useful security information without direct vulnerability

18. Secure Coding Practices

Input Validation

Validate all user-controlled input on the server side.

Parameterized SQL

Use prepared statements instead of constructing SQL queries with user input.

Output Encoding

Encode output according to its context to prevent XSS.

Authentication

Use secure password hashing and enforce proper authorization checks.

Session Security

Use secure session cookies and regenerate session IDs after login.

Error Handling

Do not expose database queries, file paths, stack traces, or internal configuration in production errors.

Access Control

Ensure users can access only the resources they are authorized to access.

Security Headers

Configure appropriate HTTP security headers.

Software Updates

Keep frameworks, libraries, web servers, and dependencies patched.

19. Screenshots to Submit

The following screenshots should be included in the practical record:

1. Target application home page
2. Nmap scan result
3. Curl HTTP-header output
4. Browser Developer Tools – Network tab
5. Burp Suite intercepted request
6. Burp Suite Repeater test
7. SQL Injection test evidence
8. XSS test evidence
9. Authentication test
10. Cookie/session analysis
11. Directory Traversal test
12. Nikto scan result
13. OWASP ZAP scan results
14. Final vulnerability findings table

20. Overall Observation

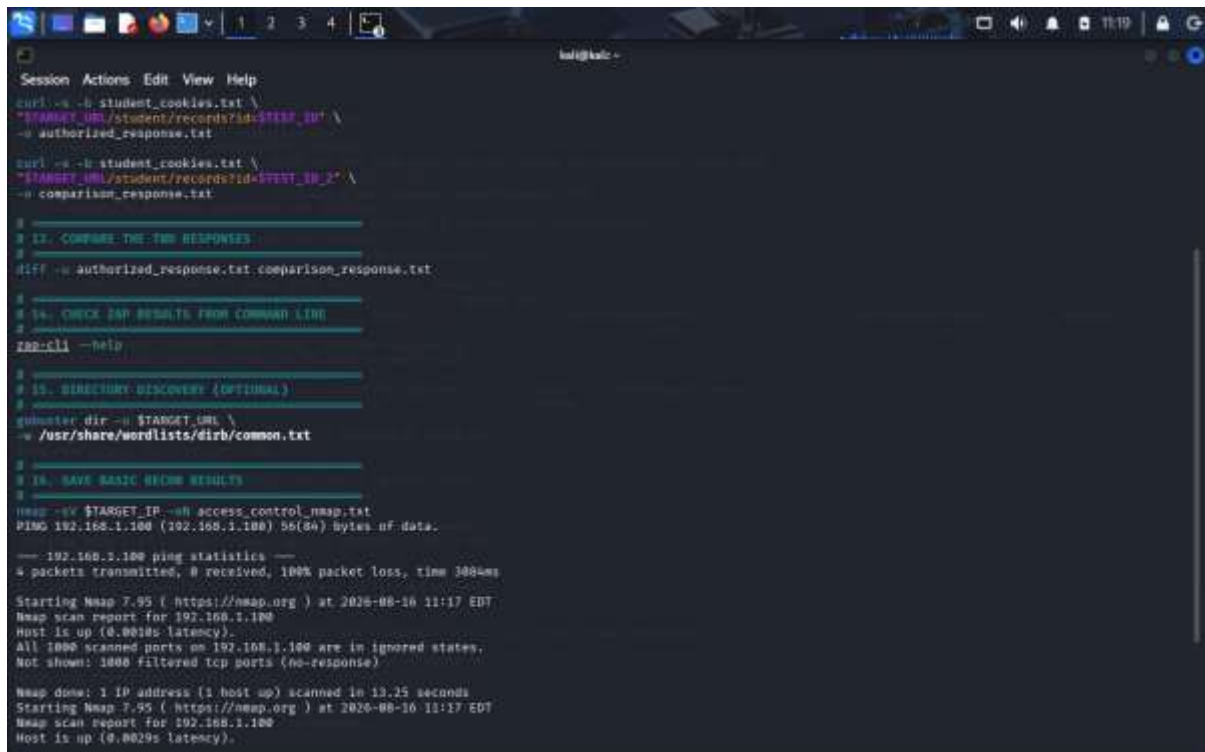
The assessment demonstrates how different security tools can be combined to analyze a web application. Nmap identifies exposed services, Curl examines HTTP responses, browser Developer Tools provide client-side request information, Burp Suite allows detailed request manipulation, OWASP ZAP performs automated application scanning, and Nikto identifies common web-server configuration problems.

The assessment should focus on confirming vulnerabilities rather than simply accepting automated scanner alerts. Each finding should be manually validated in the authorized laboratory environment and supported with evidence.

21. Conclusion

The authorized web application vulnerability assessment identified the application's attack surface and demonstrated the process of testing common vulnerabilities including SQL Injection, XSS, authentication weaknesses, session-management problems, directory traversal, and security misconfigurations.

The vulnerabilities should be prioritized according to severity and potential impact. Secure coding practices such as parameterized queries, input validation, output encoding, secure session management, strong authentication, proper access control, secure HTTP headers, and regular software updates can significantly improve the security of the web application.



```
Session Actions Edit View Help
curl -s -b student_cookies.txt \
"TARGET_URL/student/records?id=STEST_ID" \
-o authorized_response.txt

curl -s -b student_cookies.txt \
"TARGET_URL/student/records?id=STEST_ID_2" \
-o comparison_response.txt

# 13. COMPARE THE TWO RESPONSES
#
diff -u authorized_response.txt comparison_response.txt

# 14. CHECK ZAP RESULTS FROM COMMAND LINE
#
zap-cli --help

# 15. DIRBUSTERY DISCOVERY (OPTIONAL)
#
gubuster dir -u $TARGET_URL \
-u /usr/share/wordlists/dirb/common.txt

# 16. SAVE BASIC SECURE RESULTS
#
nmap -sV $TARGET_IP -oN access_control_nmap.txt
PING 192.168.1.100 (192.168.1.100) 56(84) bytes of data:

--- 192.168.1.100 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3084ms

Starting Nmap 7.95 ( https://nmap.org ) at 2020-08-16 11:17 EDT
Nmap scan report for 192.168.1.100
Host is up (0.0018s latency).
All 1000 scanned ports on 192.168.1.100 are in ignored states.
Not shown: 3000 filtered tcp ports (no-response)

Nmap done: 1 IP address (1 host up) scanned in 13.15 seconds
Starting Nmap 7.95 ( https://nmap.org ) at 2020-08-16 11:17 EDT
Nmap scan report for 192.168.1.100
Host is up (0.0029s latency).
```

Question 3: Security Header and HTTPS Configuration Assessment

A cybersecurity analyst is assigned an authorized web application hosted in an isolated laboratory environment to assess its HTTPS and security-header configuration. You are required to use Curl to examine HTTP response headers and Nmap SSL scripts to analyze the server's HTTPS/TLS configuration. Identify missing or improperly configured security headers such as Content-Security-Policy (CSP), HSTS, X-Frame-Options, and X-Content-Type-Options, and verify whether HTTP requests are automatically redirected to HTTPS. Examine the TLS configuration for weak protocols, insecure cipher suites, and certificate-related issues. Analyze the security purpose and potential impact of each misconfiguration, classify the findings according to their severity, and recommend appropriate HTTPS, TLS, certificate, and security-header hardening measures. Submit the commands used, screenshots, observations, identified risks, severity classification, and remediation recommendations.

Answer:

1. Objective

To assess the HTTPS/TLS configuration and HTTP security headers of an authorized web application and identify security misconfigurations, weak protocols, insecure cipher suites, and certificate issues.

2. Tools Used

- Curl

- Nmap
- Web Browser

3. HTTP Security Header Analysis

Check HTTP headers

```
curl -I http://<TARGET_URL>
```

Check HTTPS headers

```
curl -I https://<TARGET_URL>
```

Check HTTP to HTTPS redirection

```
curl -I http://<TARGET_URL>
```

If the response contains:

HTTP/1.1 301 Moved Permanently

Location: https://<TARGET_URL>/

then HTTP is correctly redirected to HTTPS.

Headers to check

Content-Security-Policy

Strict-Transport-Security

X-Frame-Options

X-Content-Type-Options

4. Nmap TLS/SSL Assessment

SSL enumeration

```
nmap --script ssl-enum-ciphers -p 443 <TARGET_IP>
```

Certificate information

```
nmap --script ssl-cert -p 443 <TARGET_IP>
```

Combined scan

```
nmap -sV --script ssl-cert,ssl-enum-ciphers -p 443 <TARGET_IP>
```

5. Observations

Check whether:

- HTTPS is enabled.
- HTTP automatically redirects to HTTPS.
- TLS 1.0/1.1 or other obsolete protocols are supported.
- Weak cipher suites are enabled.
- The certificate is valid and not expired.

- The certificate matches the domain name.
- Required security headers are present.

6. Vulnerability Classification

Finding	Severity	Impact
Missing CSP	Medium	Increases XSS risk
Missing HSTS	Medium	Allows possible downgrade/HTTP attacks
Missing X-Frame-Options	Medium	Increases clickjacking risk
Missing X-Content-Type-Options	Low	Allows MIME-sniffing risks
HTTP not redirected to HTTPS	High	Traffic may remain unencrypted
Weak TLS protocol	High	Reduces encryption security
Weak cipher suite	Medium/High	May weaken confidentiality
Expired/invalid certificate	High	Reduces trust and may enable interception

7. Remediation

Security Headers

Configure:

Content-Security-Policy: default-src 'self'

Strict-Transport-Security: max-age=31536000; includeSubDomains

X-Frame-Options: SAMEORIGIN

X-Content-Type-Options: nosniff

HTTPS/TLS Hardening

- Redirect all HTTP traffic to HTTPS.
- Disable TLS 1.0 and TLS 1.1.
- Use modern TLS versions such as TLS 1.2 and TLS 1.3.
- Disable weak cipher suites.
- Use a trusted, valid, non-expired certificate.
- Ensure the certificate hostname matches the application domain.
- Renew certificates before expiration.

8. Screenshots to Submit

1. curl -I HTTP output.
2. HTTPS response headers.
3. HTTP → HTTPS redirect response.
4. Nmap SSL cipher scan.

- ## 9. Conclusion

```

PS> kali@kali: /home/kali

Session Actions Edit View Help

Starting Nmap 7.95 ( https://nmap.org ) at 2025-08-16 11:14 EDT
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.02 seconds

kali@kali:~/home/kali
PS>

kali@kali:~/home/kali
PS>

kali@kali:~/home/kali
PS> # 14. SAVE RESULTS

kali@kali:~/home/kali
PS>

kali@kali:~/home/kali
PS> curl -o $HTTPS_URL | tee https_headers.txt
curl: (2) no URL specified
curl: try 'curl --help' or 'curl --manual' for more information

kali@kali:~/home/kali
PS>

kali@kali:~/home/kali
PS> nmap -sS -sV --script ssl-enum-ciphers --ssl-cert $TARGET_IP --o $ssl_results.txt
Starting Nmap 7.95 ( https://nmap.org ) at 2025-08-16 11:14 EDT
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.05 seconds

kali@kali:~/home/kali
PS>

kali@kali:~/home/kali
PS> openssl s_client -connect $TARGET_DOMAIN:443 -servername $TARGET_DOMAIN </dev/null 2>/dev/null | tee tls_connection.txt
Warning: data stream is truncated
line 1
1 - sent - connect $TARGET_DOMAIN:443 -servername $TARGET_DOMAIN </dev/null -
The 's' parameter is reserved for future use.

kali@kali:~/home/kali
PS>

```

A cybersecurity analyst is assigned an authorized student portal in an isolated laboratory environment containing separate student and administrator test accounts. Using OWASP ZAP, capture and analyze requests generated while accessing student records and identify test object identifiers in URLs, forms, cookies, or API requests. Using only the test identifiers provided by the faculty, determine whether one student account can access another student's records or whether a normal user can directly access administrator-only resources. Compare the server responses for authorized and unauthorized requests, explain the difference between authentication and authorization failures, classify the identified access-control weaknesses based on their severity and impact, and recommend appropriate server-side authorization checks, role validation, and secure object-reference mechanisms. Submit the testing procedure, commands/configuration, screenshots, observations, confirmed findings, severity,

impact, and remediation measures.

1. Objective

To test whether an authorized student portal correctly enforces access control between student and administrator accounts using OWASP ZAP.

2. Tools Used

- OWASP ZAP
- Web Browser
- Browser Developer Tools

3. Testing Procedure

Step 1: Login as Student

Login using the provided student test account and access the student's record.

In OWASP ZAP → History, identify the request containing an object identifier.

Example:

/student.php?id=101

Record the identifier provided by the faculty.

Step 2: Test Another Student ID

Using only the authorized test identifier, send the request again with the second test student's ID.

Compare:

Authorized request → Student A record

Unauthorized request → Student B record

If Student A can view Student B's information, this indicates a possible IDOR/Broken Object-Level Authorization vulnerability.

Step 3: Test Administrator Resource

Identify an administrator-only URL using the authorized laboratory application.

Example:

/admin/dashboard

While logged in as a normal student, request the administrator resource.

Observe the response.

4. Response Comparison

Test	Expected Result	Observation
Student accesses own record	Access allowed	Successful
Student accesses another student's record	Access denied	Record should not be displayed

Student accesses admin page	Access denied	403/redirect expected
Admin accesses admin page	Access allowed	Successful

A 403 Forbidden, appropriate authorization error, or safe redirect generally indicates that access control is being enforced.

5. Authentication vs Authorization

Authentication answers:

“Who is the user?”

Example: username and password verify the student's identity.

Authorization answers:

“What is this user allowed to access?”

Example: a student can access their own record but cannot access another student's record or an administrator page.

Therefore, a user can be successfully authenticated but still be unauthorized to access a particular resource.

6. Possible Findings

Finding 1: IDOR / Broken Object-Level Authorization

Observation: A student account can access another student's record by changing an authorized test object identifier.

Severity: High

Impact: Unauthorized disclosure or modification of student information.

Recommendation: Perform server-side ownership checks for every object request.

Finding 2: Vertical Privilege Escalation

Observation: A normal student account can access an administrator-only resource.

Severity: High/Critical

Impact: Unauthorized administrative actions or access to sensitive information.

Recommendation: Check the user's role and permissions on the server for every administrator request.

7. Secure Object References

Avoid relying only on predictable IDs such as:

?id=101

?id=102

?id=103

Use authorization checks even when random identifiers are used.

For example:

User → Request Object → Server checks ownership/role → Allow/Deny

Random identifiers can reduce enumeration risk, but they do not replace authorization checks.

8. Remediation

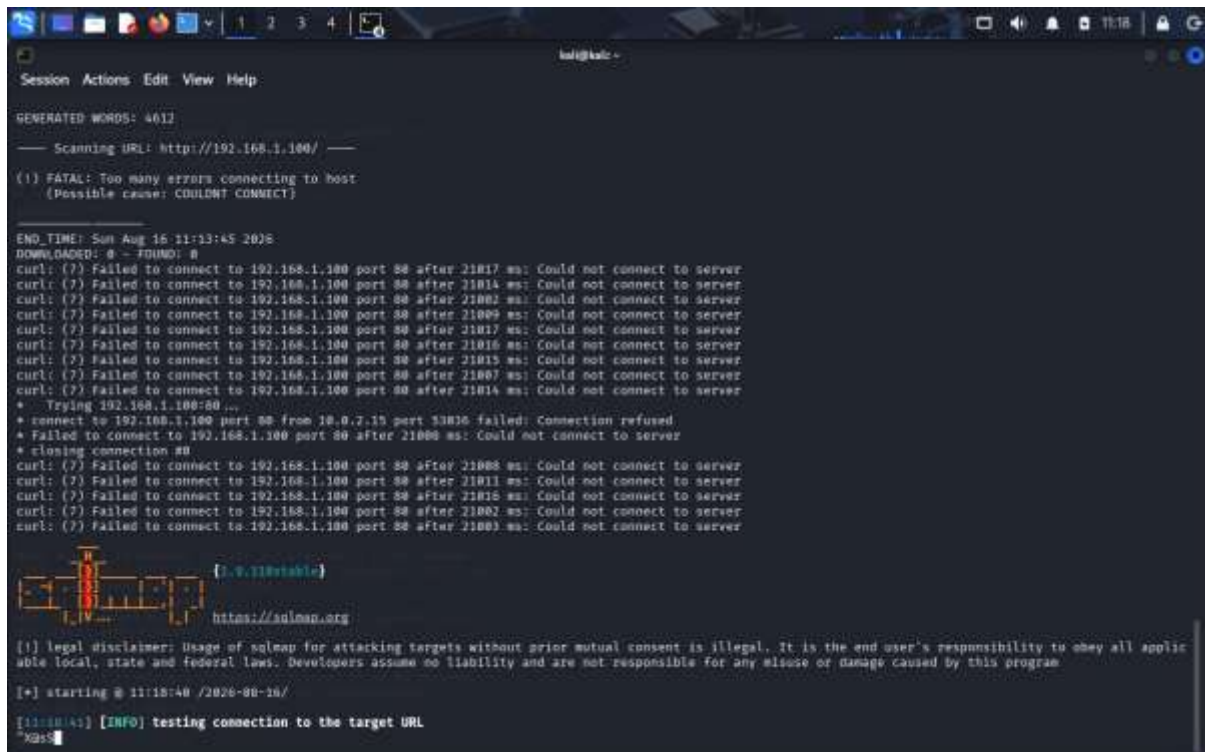
- Implement server-side authorization checks.
- Verify object ownership before returning records.
- Validate user roles for administrator resources.
- Deny unauthorized requests with appropriate responses.
- Never rely only on hidden fields, URLs, cookies, or client-side JavaScript for authorization.
- Log unauthorized access attempts.
- Use least-privilege access controls.
- Use secure, non-predictable object references where appropriate.

9. Screenshots to Submit

1. Student login.
2. Student record request in ZAP History.
3. Authorized request and response.
4. Unauthorized test request and response.
5. Administrator resource request.
6. Access-denied response.
7. Final vulnerability/severity table.

10. Conclusion

The assessment verifies whether the student portal correctly separates student and administrator permissions. A confirmed vulnerability occurs when a student can access another student's record or administrator functionality without proper authorization. Strong server-side ownership checks, role validation, and least-privilege access control are required to prevent broken access-control vulnerabilities.



```
Session Actions Edit View Help
GENERATED WORDS: 4612
--- Scanning URL: http://192.168.1.100/ ---
(1) FATAL: Too many errors connecting to host
(Possible cause: COULDN'T CONNECT)

END_TIME: Sun Aug 16 11:13:45 2026
DOWNLOADED: 0 - FOUND: 0
curl: (7) Failed to connect to 192.168.1.100 port 80 after 21817 ms: Could not connect to server
curl: (7) Failed to connect to 192.168.1.100 port 80 after 21814 ms: Could not connect to server
curl: (7) Failed to connect to 192.168.1.100 port 80 after 21802 ms: Could not connect to server
curl: (7) Failed to connect to 192.168.1.100 port 80 after 21809 ms: Could not connect to server
curl: (7) Failed to connect to 192.168.1.100 port 80 after 21817 ms: Could not connect to server
curl: (7) Failed to connect to 192.168.1.100 port 80 after 21816 ms: Could not connect to server
curl: (7) Failed to connect to 192.168.1.100 port 80 after 21815 ms: Could not connect to server
curl: (7) Failed to connect to 192.168.1.100 port 80 after 21807 ms: Could not connect to server
curl: (7) Failed to connect to 192.168.1.100 port 80 after 21814 ms: Could not connect to server
* Trying 192.168.1.100:80...
* connect to 192.168.1.100 port 80 from 10.0.7.15 port 53836 failed: Connection refused
* Failed to connect to 192.168.1.100 port 80 after 21806 ms: Could not connect to server
* closing connection #0
curl: (7) Failed to connect to 192.168.1.100 port 80 after 21808 ms: Could not connect to server
curl: (7) Failed to connect to 192.168.1.100 port 80 after 21811 ms: Could not connect to server
curl: (7) Failed to connect to 192.168.1.100 port 80 after 21815 ms: Could not connect to server
curl: (7) Failed to connect to 192.168.1.100 port 80 after 21802 ms: Could not connect to server
curl: (7) Failed to connect to 192.168.1.100 port 80 after 21803 ms: Could not connect to server

[1.9.118table]
https://sulemns.org

(1) legal disclaimer: Usage of sulemns for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 11:15:40 /2026-08-16/
[11:15:41] [INFO] testing connection to the target URL
XGASG
```

Question 5: Automated Scanning and Manual Validation

A cybersecurity analyst is assigned an authorized web application hosted in a controlled cybersecurity laboratory for vulnerability assessment. Use Nikto to identify web-server misconfigurations and outdated components, and use Wapiti or OWASP ZAP to perform an automated web-application security scan. Compare the vulnerabilities reported by the selected tools and categorize them into misconfiguration, information disclosure, authentication, and injection issues. Select at least three findings for safe manual validation within the authorized laboratory environment and determine whether each finding is confirmed, a false positive, or requires further investigation. Assign an appropriate severity level based on likelihood and potential impact, and recommend practical remediation measures for all confirmed vulnerabilities. Submit a comparative report containing tool outputs, screenshots, validation evidence, severity classification, remediation measures, and overall conclusions.

Answer:

1. Objective

To identify web-server and web-application vulnerabilities using Nikto and OWASP ZAP, compare their findings, manually validate selected findings, classify their severity, and recommend suitable remediation.

2. Tools Used

- Nikto
- OWASP ZAP
- Web Browser
- Curl

3. Nikto Scan

Run Nikto against the authorized laboratory application:

```
nikto -h <TARGET_URL>
```

Observe

Nikto may identify:

- Missing security headers
- Server information disclosure
- Directory listing
- Backup files
- Outdated components
- Insecure configuration

Take a screenshot of the scan output.

4. OWASP ZAP Scan

Open OWASP ZAP → Quick Start → Automated Scan.

Enter:

```
<TARGET_URL>
```

Start the scan and review the alerts under the Alerts tab.

Record:

- Alert name
- URL
- Parameter
- Risk level
- Evidence
- Recommended solution

5. Comparison of Tools

Category	Nikto	OWASP ZAP
----------	-------	-----------

Web-server misconfiguration	Strong	Yes
-----------------------------	--------	-----

Category	Nikto	OWASP ZAP
Information disclosure	Yes	Yes
Authentication issues	Limited	Better coverage
Injection issues	Limited	Better coverage
Security headers	Yes	Yes
Hidden/dangerous files	Strong	Limited
Web-application testing	Basic	Strong

Observation: Nikto is mainly useful for web-server configuration and known dangerous files, while ZAP provides broader web-application vulnerability testing.

6. Vulnerability Categorization

Finding	Category	Tool	Severity
Missing security headers	Misconfiguration	Nikto/ZAP	Low/Medium
Server version disclosure	Information Disclosure	Nikto	Low
Exposed backup file	Information Disclosure	Nikto/ZAP	High
Weak authentication	Authentication	ZAP	High
SQL Injection alert	Injection	ZAP	High/Critical
XSS alert	Injection	ZAP	Medium/High

Only include findings actually reported by the tools.

7. Manual Validation

At least three findings should be manually checked in the authorized laboratory environment.

Finding 1: Missing Security Header

Test

```
curl -I <TARGET_URL>
```

Check whether the following header is missing:

Content-Security-Policy

Result

If the header is genuinely absent:

Status: Confirmed

Severity: Medium/Low

Impact: Provides less protection against certain browser-based attacks.

Remediation

Configure an appropriate Content-Security-Policy header.

Finding 2: Information Disclosure

Test

```
curl -I <TARGET_URL>
```

Check for:

Server: Apache/2.4.x

X-Powered-By: PHP/8.x

Result

If unnecessary version information is exposed:

Status: Confirmed

Severity: Low

Impact: Helps attackers identify technologies and potentially vulnerable versions.

Remediation

Hide unnecessary server and framework version information.

Finding 3: Injection Alert

If ZAP reports an injection vulnerability, capture the affected request and inspect the parameter.

Using ZAP → History/Request Editor, compare the normal request with the scanner's test request.

Result

- If the vulnerability can be reproduced safely → Confirmed
- If the application behaves normally → False Positive
- If there is insufficient evidence → Requires Further Investigation

Severity

Usually High/Critical if actual unauthorized database or command execution is demonstrated.

Remediation

- Validate input.
- Use parameterized queries.
- Avoid dynamically constructed SQL commands.
- Apply least-privilege database permissions.

8. Manual Validation Results

Finding	Validation Result	Severity	Impact
Missing CSP	Confirmed	Medium	Reduced protection browser

Server information disclosure	Confirmed	Low	Assists reconnaissance
Injection alert	Confirmed/False Positive/Investigate	High/Critical	Possible data compromise

Replace the result with the actual outcome obtained during testing.

9. Remediation Measures

Misconfiguration

- Apply security headers.
- Disable directory listing.
- Remove unnecessary services.
- Keep server software updated.

Information Disclosure

- Hide server versions.
- Remove backup and temporary files.
- Avoid exposing configuration files.
- Configure safe error messages.

Authentication

- Use strong authentication.
- Enforce authorization checks server-side.
- Implement appropriate session management.
- Apply rate limiting where appropriate.

Injection

- Use prepared statements.
- Validate and sanitize input.
- Use parameterized queries.
- Apply least-privilege database accounts.

10. Screenshots to Submit

1. Nikto command and output.
2. OWASP ZAP scan configuration.
3. ZAP Alerts page.
4. Curl response-header output.
5. Manual validation of Finding 1.
6. Manual validation of Finding 2.

- ## 11. Conclusion

Manual validation is important because automated scanners can produce false positives. The selected findings should therefore be tested safely in the authorized laboratory environment and classified as confirmed, false positive, or requiring further investigation. Confirmed vulnerabilities should be prioritized according to their likelihood and potential impact and remediated using secure configuration and secure coding practices.

[illegible]